DriX-8 Fuel Efficiency Analysis

Data Sources and Calculation Methods

EM2040 gondola · 6 days of MCAP logs · 4.9 h of steady cruise

13 August 2026

What this document is for

It records which ROS 2 topics the endurance numbers come from and exactly how those numbers are produced — extraction, segmentation, binning, fitting and the derived quantities — so any figure can be traced back to a channel and a decision. Every value here is read from the pipeline output at build time; none is typed by hand. It is the companion to the main fuel-efficiency report, which covers findings rather than method.

1. Data sources

All analysis derives from the vehicle's connectivity-box MCAP recordings — the same bags the ROS 2 topic reference documents. Nothing is taken from operator notes, and no value is entered by hand at any stage of the pipeline.

Day	Telemetry samples	Span (h)	Engine on	GPS	INS	Trim
2026-08-04	101,104	16.6	1%	68,418	42,099	—
2026-08-05	77,221	10.9	50%	96,562	32,303	—
2026-08-06	90,724	13.0	50%	113,227	37,843	—
2026-08-07	81,340	12.6	0%	101,670	33,884	—
2026-08-08	128,268	19.0	30%	159,289	53,397	24,648
2026-08-09	166	0.0	0%	84	69	36

Table 1 — Extracted cache per day. "Engine on" is the fraction of telemetry samples with the engine running; days near zero are alongside days that contribute only to the gauge and idle records.

Segment numbering is not always contiguous — recordings have been observed missing a segment mid-day. The pipeline sorts by timestamp rather than filename and only ever uses windows of continuous data, so a gap removes coverage but cannot corrupt a fit. It does mean whole-day totals understate elapsed time, and any per-day fuel arithmetic must not assume continuity across a gap.

2. Topics used

Six topics carry every number in the endurance analysis. They were chosen to be the minimum sufficient set: decoding is the dominant cost of the pipeline, so channels that do not change an answer are not read.

Topic	Message type	Fields consumed	Used for
…/plc/telemetry/telemetry3	drix_msgs/Telemetry3	fuel_rate_lph, engine_on, total_fuel_used_l, alternator_1/2_current	The fuel measurement itself; engine-running gate; integral cross-check
…/plc/vehicle_status	drix_msgs/DrixOutput	thruster_rpm, gasoline_level_percent	Engine RPM (see §2.1); tank gauge for the capacity calibration
…/topic_simplifier/light_gps	mdt_msgs/LightGps	sog (m/s), cog, latitude, longitude	Speed over ground and course steadiness
…/topic_simplifier/light_ins	cortix_msgs/LightIns	heave, pitch, roll, heading	PHINS-derived motion statistics — the sea-state proxy
…/hardware/trimmer/status	drix_msgs/TrimmerStatus	position_deg, zero_position_deg	Trim-tab attitude, a drag variable under watch
…/fuel_cons_manager/fuel_consumption	cortix_msgs/FuelConsumption	consumption_4/8/12kn, dynamic_model_used	Cross-check against the vehicle's own static model

Table 2 — Topics read by tools/extract_bags.py. INS is subsampled 1-in-3 and the Exail model 1-in-10; the rest are taken at full rate.

2.1 Why thruster_rpm and not shaft_sensor_rpm

Telemetry3 publishes a shaft RPM sensor reading, which would be the natural choice. It is faulted throughout the recorded period — reading zero or 65527 — so the PLC's thruster_rpm from vehicle_status is used instead. This is sound rather than a compromise: the DriX-8 drivetrain is direct drive, so shaft RPM equals engine RPM (the sole exception, the trawling motor, is no longer used). The extraction still records the faulted channel so the fault remains visible in the cache.

2.2 Topics deliberately not used

- **Weather sensor — not available.** A sweep of every channel in every segment found no live wind or meteorological topic. The only weather-adjacent message is a blank operator form. The extraction probes each new day's first segment against a wishlist and announces any arrival, so the moment the sensor is bridged into the recording the pipeline will say so.
- **Raw PHINS / high-rate IMU — not recorded.** Only the derived light_ins stream is bridged. It is sufficient for motion statistics at the timescales that matter here, but a high-rate attitude stream would sharpen them.
- **Radar, lidar, AIS, diagnostics, bandwidth.** High message volume, no bearing on fuel. Excluded to keep decode time proportionate.

3. Extraction

```
python tools/extract_bags.py [BAG_ROOT]
```

Bags → one compressed NPZ cache per day in tools/rosbags/. Existing caches are skipped, so adding a day costs only that day's decode.

Three details matter for correctness:

- **Segment ordering.** Files sort lexically, which places segment _10 before _2. The extractor sorts numerically and the fitting stage additionally sorts every channel by timestamp on load.
- **Units.** SOG arrives in metres per second and is converted to knots with the exact factor 3600/1852; RPM and fuel rate are used as published.
- **Cache integrity.** A day folder containing no segments raises rather than writing an empty cache — this catches a bag root pointed one level too high, which otherwise fails silently.

4. Segmentation

A fuel curve requires steady state. Instantaneous flow-meter readings during acceleration, turns or manoeuvring describe transients, not the relationship between engine speed and consumption, so the pipeline selects only windows where the vehicle is genuinely settled.

4.1 Cruise

A sample qualifies as cruise when, over the preceding 60 seconds, all of the following hold:

Condition	Threshold	Why
Engine running	engine_on true	Excludes drifting and alongside time
RPM steady	rolling std ≤ 40 rpm	Rejects acceleration and throttle changes
Speed steady	rolling std ≤ 0.20 m/s	Rejects surging and wake effects
Course steady	rolling std ≤ 6° (unwrapped)	Rejects turns — a turning hull carries rudder drag that is not cruise drag
RPM in band	1200 ≤ rpm < 3200	Above idle, below the recorded ceiling
Making way	sog > 1.5 m/s	Excludes station-keeping

Table 3 — Cruise criteria. Course unwrapping matters: without it a heading crossing 360° reads as a 360° excursion and every northerly leg would be discarded.

4.2 Loiter

Station-keeping is retained separately — engine on, 800–1200 rpm, fuel rate above 0.3 L/h. It is deliberately excluded from the cruise fit: at those revolutions the propeller is loaded quite differently and the vehicle is not making way, so including it would bend the cruise curve toward a regime it does not describe. It is reported on its own as the idle and station-keeping burn.

5. Binning and fitting

Steady samples are pooled across all days and grouped into 100-rpm bins. A bin is used only if it holds at least 120 samples — about two minutes of steady running — and each bin contributes its median speed and median fuel rate. Medians rather than means because a single flow-meter spike should not move a bin.

Fits are weighted least squares over the bin medians, with weights equal to sample count capped at 3600. The cap is the point: without it, one long leg at a favourite cruise setting would dominate the entire curve, and the fit would describe that leg rather than the vehicle.

```
speed:  KNOTS = b + m · RPM              (linear)
fuel :  L/H   = q0 + q1 · RPM + q2 · RPM²  (quadratic)
```

The quadratic fuel form is used because a propeller absorbs torque roughly as the square of shaft speed; the linear speed form matches the observed data across the cruise band.

5.1 The laws this produced

For the record, the coefficients currently in force — read from the pipeline output, not transcribed:

```
KNOTS = 0.486068 + 0.00344786 · RPM
```

R² 0.9558 over 12 bins.

```
L/H   = 2.936436 -0.00297043 · RPM +1.458113e-06 · RPM²
```

R² 0.9995, valid 1467–3079 rpm.

At the 8 kt survey speed that is 2179 rpm, 3.39 L/h, 2.36 NM/L — inside the fitted window, so survey planning on this configuration is interpolation rather than extrapolation.

RPM bin	Samples	SOG (kt)	Fuel (L/h)	NM/L
1400–1499	1,264	6.02	1.70	3.54
1500–1599	2,301	5.92	1.85	3.20
1600–1699	5,678	6.10	1.95	3.13
1700–1799	685	6.33	2.35	2.69
1800–1899	5,003	6.74	2.45	2.75
1900–1999	397	7.27	2.75	2.64
2000–2099	337	7.10	3.05	2.33
2100–2199	140	8.04	3.45	2.33
2200–2299	508	7.97	3.60	2.21
2300–2399	143	8.01	3.85	2.08
2500–2599	148	8.51	4.58	1.86
3000–3099	614	11.73	7.62	1.54

Table 4 — The bin medians the fits are computed over. Bin count and contents change as data accumulates; this table is regenerated with the document.

5.2 Per-day agreement

Because data now arrives daily, the fit reports each day's bin medians against the pooled result. A day that disagrees materially is named with its worst bin rather than being quietly averaged into the pool — the check that would catch a fouled hull, a changed payload, or a sensor drifting out of calibration.

6. Derived quantities

6.1 Efficiency, range and endurance

```
efficiency  (NM/L) = KNOTS / (L/H)
range       (NM)   = mission fuel × efficiency
endurance   (h)    = mission fuel ÷ (L/H)
mission fuel (L)   = ∫ gauge, from the reserve floor up to the start level
```

NOT capacity × (1 − reserve fraction). That formula was retired in v2.4.0: it assumes the gauge is linear and that a tank percentage is a fixed number of litres, and neither holds.

Planning figures are computed to the 25% return-to-port reserve rather than to a dry tank. Tank volume is ESTABLISHED at 250 L from the engineering drawings, but that is not the same quantity as mission fuel: the floor is written in indicated percent, so what a mission may spend is an integral over the gauge between the start level and the floor. See the fuel-efficiency report §8.2 and the gauge-linearity report for why those two differ by about 44 L.

The difference is not academic. On the adopted reading the integral gives 185.7 L from a full tank, where capacity × (1 − reserve) would give 187.5 L. Those happen to sit close together because the profile re-normalises to the drawing volume; on the conservative reading (B) the same two figures are 154.5 L and 187.5 L, which is a mission-shaping difference. The integral is the quantity the planner uses in both cases.

6.2 Gauge calibration

The tank gauge is calibrated directly against the flow meter: total metered litres over a day divided by the indicated percentage points consumed, using ten-minute medians at each end so a single noisy reading cannot set an endpoint. Days that burn less than two litres are excluded — an idle day's gauge wanders a point or two on essentially no fuel, and letting one anchor the span moves the answer by around a tenth for no physical reason.

This is the only non-circular capacity measurement available: it assumes no tank volume and no fuel model. The result is band-specific, and the gauge is not linear, so a figure measured in one part of the range must not be extrapolated across the tank.

6.3 Through-origin variants for ops tables

The planner uses the fitted laws as measured, because they are most accurate in the band where missions are actually flown. An operator-facing endurance table has a different requirement: it must behave physically at its ends, showing zero fuel at zero RPM. For those tables the same bin data is refitted under a through-origin constraint, anchored additionally by the measured idle burn. The two families agree closely in band, and the endurance sheet states which it uses.

7. Currents from the NOAA forecast

A second data source, acquired and used differently from everything above. The endurance laws come from the vehicle's own recordings; the tidal current comes from an operational model published by NOAA, and it is the only part of the planner that touches a network. It is documented here because it is an input to a plan, and an input a reader may need to defend.

What it does NOT change

No coefficient moves. model.json is untouched by any of this, the engine gained nothing, and no fitted value is involved. The forecast supplies two inputs that already existed — a leg's current speed and set — so an operator who types the tide in from a table gets exactly the same arithmetic. Never pressing the button leaves the planner as it was.

7.1 Product, and the decision that mattered

NOAA publishes the Delaware Bay Operational Forecast System over OPeNDAP in two forms. The native ROMS output carries velocities on a curvilinear grid in GRID axes on staggered points: using it means averaging u and v onto rho points and rotating every cell by its own angle before a bearing means anything. The regridded product is rectilinear and already true-referenced.

This reads the regridded product, because the two operations the native one needs are exactly the two that fail SILENTLY — get the rotation wrong and every set is out by tens of degrees while every speed still looks reasonable. That shortcut is therefore evidenced rather than assumed: currents.py crosscheck performs the native path by hand and compares, and the two agree to a median 0.07 kt and 1.0°.

7.2 What a cycle is, and which one answers

Property	Value
Nowcast hours (up to the cycle time)	6
Forecast hours (after it)	48
Frames per cycle	54 hourly
Span	53 h
Layer	surface only (depth 0 m)
Archive NOAA serves	about 2 days
Projection reach	3 tidal cycles (37.3 h)

Table 5 — Cycle shape and the limits around it, read from currents.py at build time.

A requested time is answered by a ladder, and real data always beats an estimate: a cached cycle covering the window; else a cycle NOAA still serves covering it, found from the catalog for the price of one page rather than a 33 MB download, and fetched; else a projection; else a refusal. Cycle selection checks the BOX as well as the span — a cycle fetched for one operating area cannot answer for another, and without that check it would report no water on every leg, which reads like a forecast of slack water rather than the wrong file.

7.3 Projection, and what it is worth

The current here is semidiurnal, so a time the forecast cannot reach is estimated from the value one M2 period (12.4206 h) away — never by holding the last value, and never by extrapolating a line through a reversing tide. That is a measurement rather than a preference:

Method	RMS error against the model
Project 1 cycle (12.4 h)	0.19 kt

Project 2 cycles (24.8 h)	0.14 kt
Project 3 cycles (37.3 h)	0.21 kt
Hold the last value	0.57 – 2.21 kt
Assume slack water	0.36 – 1.46 kt

Table 6 — Measured on dbofs_20260813_t00z across 258 samples (2026-08-13), reproducible with tools/projection_accuracy.py. Flat out to three cycles because a tide repeats rather than decays. At the bay entrance, where it runs to 2.2 kt, a projection lands within 0.27 kt while persistence is wrong by the whole tide.

The reach is capped at 3 cycles for that reason and not because it is a round number: the error is flat that far because a tide repeats, but the non-tidal part — wind setup, river flow — does not repeat at all, and beyond about a day and a half there is no evidence in hand. Two things a projection never does: it never moves a POSITION, so a point with no model water is unanswered however the time is shifted; and it never happens silently — a projected leg is flagged in the response, named in its note, marked in the provenance label the mission report prints, and surfaced in the plan warnings.

7.4 How a current reaches a plan

There are exactly two seams, and they differ in resolution rather than in kind:

- **Per leg, one number each.** The mission is dead-reckoned from the departure position and time, each leg sampled along its own track at the time the vehicle would be there, and the samples vector-averaged. A survey holds position and only advances the clock, because a lawnmower ends roughly where it began and walking it down the first line's course would put the run home in the wrong water.
- **Along the track, a field.** Given geometry, the planner instead calls back once per RUN — per survey line, per transit segment — with where the vehicle is and how far into the mission, and takes the current there and then. This is what makes a turning tide real rather than averaged.

Either way the current changes the FUEL and never the clock: the required speed over ground is converted to the speed through the water the hull must actually make, and that goes through the gondola's speed law to get RPM. A leg still takes distance divided by speed.

> **It is partly counted twice, and the leg note says so**
>
> The speed law is fitted against speed over ground in an unrecorded tide, so some tidal effect is already inside it. Reading the tide off the forecast makes the INPUT better; it does not make the correction clean. The honest framing is "compare two plans" or "price today's tide", not "a calibrated tidal model". Removing it properly needs the same reciprocal-heading runs §9 already asks for.

7.5 A worked example

One mission, planned three ways, against a real cycle. Twelve miles out on 045, a 12 × 2 NM survey on 020, twelve miles home on 225, departing Lewes at midday UTC:

Planned with	Fuel	Mission clock
Slack water (no current)	17.18 L	7.43 h
Per-leg currents from the forecast	18.08 L (+5.2%)	7.43 h
Sampled along the track	17.64 L (+2.7%)	7.43 h

Table 7 — Transcript against dbofs_20260813_t00z. The clock is identical in all three: a current moves the fuel and never the time.

The reading that matters is the third row. Sampling along the track gives 2.7% where the per-leg average gives 5.2% — nearly double — because the survey sits on one ground for four hours while the tide turns under it, and a single vector average of a reversing current overstates what the boat actually fights. That is the argument for entering geometry, in numbers rather than in principle.

8. Verification

- **Flow-meter integral against the vehicle's own counter.** Integrated fuel rate is compared with the cumulative total_fuel_used_l for each day; agreement confirms neither channel is drifting.
- **Per-day agreement.** Described in §5.1 — the standing check as data accumulates.
- **Adoption comparison.** tools/compare_fits.py contrasts a fresh fit with the coefficients currently adopted and reports operational deltas — burn at survey speed, planning range, endurance — rather than raw coefficients, which are not interpretable in isolation.
- **Mutation tests.** The planner's test suite perturbs each coefficient and asserts the results move. A model change that no test notices means the tests are broken, not that the change is safe.
- **Currents, three independent ways.** Direction fails quietly, so the forecast path is checked against things that are not it: currents.py crosscheck reproduces the native ROMS path by hand (median 0.07 kt, 1.0°); tools/dbofs_plotcheck.py redraws the extracted field onto NOAA's own published image, deriving the georeference from the plot's graticule rather than a stated corner; and currents.py station_check compares against CO-OPS harmonic predictions at a real station (r = +0.987 over 54 h).
- **Projection accuracy.** tools/projection_accuracy.py re-measures what a projection is worth against the model's own output, and reports whether the recorded block still matches. §7.3 quotes that block rather than a typed number.

9. Limits of the method

Stated plainly, because each one bounds how far the numbers should be trusted:

- **Speed is over ground.** No speed-through-water sensor is recorded, so tidal set enters the speed law directly. Reciprocal-heading runs at matched RPM would remove it; until then the speed law carries a few percent of tidal uncertainty and the endurance sheet says so.
- **Sea state is a proxy.** Motion statistics from the derived INS stream stand in for sea state. They measured the calm anchor — no fuel premium above the noise floor in near-glassy water — but the slope into rough water is unmeasured because no rough-water cruise data exists yet.
- **Below the cruise band there is only the idle point.** No steady cruise data exists below the fitted floor; the region between idle and that floor is bridged by curve shape, not measurement.
- **One vehicle, one gondola configuration.** These laws describe DriX-8 as currently fitted. Fouling, loading and payload changes all move them, which is exactly why per-day agreement is monitored.
- **The forecast current is surface only, and partly double-counted.** It is the 0 m layer on an hourly grid a few hundred metres across; no shear is modelled, and the vehicle draws two metres. Because the speed law above is fitted against speed over ground in an unrecorded tide, applying a forecast current counts some of that tide twice — see §7.4. It improves the input; it does not make the correction clean.
- **The forecast archive is about two days deep.** Beyond it a requested time can only be estimated, and only within 3 tidal cycles (37 h) of data actually held. Past that the planner declines rather than guessing, and a position with no model water is never answered at any time by any route.

Appendix — Reproducing this analysis

The whole chain, in order. Extra dependencies are numpy, mcap and mcap-ros2-support; the planner itself remains standard-library only.

```
python tools/extract_bags.py D:/Claude/Fuel/D8_2040
python tools/fit_em2040.py
python tools/compare_fits.py
python tools/build_endurance_sheet.py  [OUT.xlsx]
python tools/build_methods_doc.py      [OUT.docx]
powershell -File tools/export_pdf.ps1  [-Path docs\X.docx]

# §7 only, and independent of the bags:
python currents.py fetch
python currents.py verify · crosscheck · station_check
python tools/projection_accuracy.py
```

Extraction skips days already cached, so adding a day costs only that day. compare_fits reports what a new fit would change before anything is adopted.

Adoption is a deliberate step, not an automatic one: a fresh fit is compared against the coefficients in force, and only if the operational deltas justify it are model.json, the test reference values and the generated documents updated together in a single commit.